

LC 1959: Minimum Total Space Wasted With K Resizing Operations

Authored by: Arpit Raj, LNMIIT Jaipur | August 27, 2026

1. Problem Understanding

Provided with:

- $nums[i]$ → no. of elements in array at time i
- k → max no. of times we can resize arrays

At a time t , size of array $size_t$ must be at least $nums[t]$.

$$\text{Space wasted} = size_t - nums[t] \geq 0$$

Need: Minimum total space wasted

Note: At start, array can have any size.

2. Examples

Example 1

Array: $[10, 20, 30]$, $k = 1$

Let initial size be = 20
and we'll resize it to 30 at $t = 2$.

Then space waste:
 $(20 - 10) + (20 - 20) + (30 - 30)$
 $= 10 + 0 + 0 = 10$

Example 2

Array: $[10, 20, 15, 30, 20]$, $k = 2$

Let sizes be:
 $[10, 20, 20, 30, 30]$

Then loss:
 $0 + 0 + 5 + 0 + 10 = 15$

3. Core Intuition

Hence this question is basically like:

- Make $(k + 1)$ parts
- Replace each number in a part by **max number in that part**

To find the answer:

1. Find sum of sizes (minimum possible)
2. Find sum of original array
3. **Subtract and return** the difference (which is the minimum possible waste).

4. C++ Implementation

```
class Solution {
public:
    int minSpaceWastedKResizing(vector<int>& nums, int k) {

        int n = nums.size();
        k++; // k operations means k+1 parts
        vector<vector<int>> rangemax(n+1, vector<int>(n+1, INT_MAX));
        vector<vector<int>> dp(n+1, vector<int>(k+1, INT_MAX));
        dp[0][0] = 0;

        // dp[i][k] for partition dp
        // see notes for intuition

        for(int l=0; l<n; l++) {
            int maxu = INT_MIN;
            for(int r=l; r<n; r++) {
                maxu = max(maxu, nums[r]);
                rangemax[l][r] = maxu;
            }
        }

        for(int i=1; i<=n; i++) {
            for(int parts=1; parts<=k; parts++) {
                for(int j=parts-1; j<i; j++) {
                    int maxuu = rangemax[j][i-1];

                    if(dp[j][parts-1] == INT_MAX) continue;
                    int curr = dp[j][parts-1] + (maxuu * (i-j));

                    dp[i][parts] = min(dp[i][parts], curr);
                    // need to minimize the sum of partitioned array
                }
            }
        }

        int origsum = accumulate(nums.begin(), nums.end(), 0);
        return dp[n][k] - origsum;
    }
};
```